

Header And Logo



Peripheral Links

[Donate to FreeBSD](#)

Search

Search

Site Navigation

- [Home](#)
- [About](#)
 - [Introduction](#)
 - [Features](#)
 - [Advocacy](#)
 - [Marketing](#)
- [Get FreeBSD](#)
 - [Release Information](#)
 - [Release Engineering](#)
- [Documentation](#)
 - [FAQ](#)
 - [Handbook](#)
 - [Porter's Handbook](#)
 - [Developer's Handbook](#)
 - [Manual Pages](#)
 - [Documentation Project Primer](#)
 - [All Books and Articles](#)
- [Community](#)
 - [Mailing Lists](#)
 - [Forums](#)
 - [User Groups](#)
 - [Events](#)
- [Developers](#)
 - [Project Ideas](#)
 - [GIT Repository](#)
- [Support](#)
 - [Vendors](#)
 - [Security Information](#)
 - [Bug Reports](#)
 - [Submit Bug-report](#)
- [Foundation](#)
 - [Donate](#)

FreeBSD Manual Pages

gpart

man

apropos

All Sections

FreeBSD 15.1-RELEASE and Ports.quarterly

html

[home](#) | [help](#)

gmap(1)

Scotch user's manual

gmap(1)

NAME

gmap, **gpart** - compute static mappings and partitions sequentially

SYNOPSIS

gmap [*options*] [*gfile*] [*tfile*] [*mfile*] [*lfile*]

gpart [*options*] [*nparts*] [*gfile*] [*mfile*] [*lfile*]

DESCRIPTION

The **gmap** program computes, in a sequential way, a static mapping of a source graph onto a target graph.

The **gpart** program is a shortcut of **gmap** for computing unweighted partitions of a source graph.

Source graph file *gfile* can only be a centralized graph file. For **gmap**, the target architecture file *tfile* describes either algorithmically-coded topologies such as meshes and hypercubes, or decomposition-defined architectures created by means of the [amk_grf\(1\)](#) program. The resulting mapping is stored in file *mfile*. Eventual logging information (such as the one produced by option **-v**) is sent to file *lfile*. When file names are not specified, data is read from standard input and written to standard output. Standard streams can also be explicitey represented by a dash '-'.

When the proper libraries have been included at compile time, **gmap** and **gpart** can directly handle compressed graphs, both as input and output. A stream is treated as compressed whenever its name is postfixed with a compressed file extension, such as in 'brol.grf.bz2' or '-.gz'. The compression formats which can be supported are the bzip2 format ('.bz2'), the gzip format ('.gz'), and the lzma format ('.lzma', on input only).

OPTIONS

- copt

Choose default mapping strategy according to one or several *options* among:

b

enforce load balance as much as possible.

q

privilege quality over speed (default).

s

privilege speed over quality.

t

enforce safety.
- h

Display some help.
- mstrat

Use sequential mapping strategy strat (see Scotch user's manual for more information).
- V

Display program version and copyright.
- vverb

Set verbose mode to verb. It is a set of one of more characters which can be:

m

mapping information.

s

strategy information.

t timing information.

TARGET ARCHITECTURES

Target architectures represent graphs onto which source graphs are mapped. In order to speed-up the obtainment of target architecture topological properties during the computation of mappings, some classical topologies are algorithmically coded into the mapper itself. These topologies are consequently simply defined by their code name, followed by their dimensional parameters:

cmplt *dim*
unweighted complete graph of size *dim*.

cmpltw *dim w0 w1 ... wdim-1*
weighted complete graph of size *dim* and of respective loads *w0*, *w1*, ..., *wdim-1*.

hcub *dim*
hypercube of dimension *dim*.

leaf *hgt n0 w0 ... nhgt-1 whgt-1*
tree-leaf graph of height *hgt* with (*n0* times *n1* times ... *nhgt-1*) vertices, with inter-cluster link weights of *w0*, *w1*, ... *whgt-1*.

mesh2D *dimX dimY*
2D mesh of *dimX* times *dimY* nodes.

mesh3D *dimX dimY dimZ*
3D mesh of *dimX* times *dimY* times *dimZ* nodes.

torus2D *dimX dimY*
2D torus of *dimX* times *dimY* nodes.

torus3D *dimX dimY dimZ*
3D torus of *dimX* times *dimY* times *dimZ* nodes.

Other target topologies can be created from their source graph description by using the [amk_grf\(1\)](#) command. In this case, the target description will begin with the code name **deco**.

MAPPINGS

Mappings are represented by as many lines as there are vertices in the source graph. Each of these lines is made of two figures: the number of the vertex (or its label if source graph vertices are labeled) and the index of the target vertex to which it has been assigned. Target vertex indices range from 0 to the number of vertices in the target architecture (that is, the number of parts) minus one.

This block of lines is always preceded by the number of such lines. In most cases, since full mappings are requested, the number of lines is equal to the number of vertices in the source graph.

EXAMPLES

Run **gpart** to compute a partition into 7 parts of graph 'brol.grf' and save the resulting ordering to file 'brol.map'.

```
$ gpart 7 brol.grf brol.map
```

Run **gmap** to compute a partition, into 3 parts of respective weights 1, 2 and 4, of graph 'brol.grf' and save the resulting mapping to file 'brol.map'. The dash '-' standard file name is used so that the target architecture description is read from the standard input, through the pipe, as provided by the 'echo' shell command.

```
$ echo "cmpltw 3 1 2 4" | gmap brol.grf - brol.map
```

SEE ALSO

[amk_grf\(1\)](#), [acpl\(1\)](#), [gmtst\(1\)](#), [dgmap\(1\)](#).

Scotch user's manual.

AUTHOR

Francois Pellegrini <francois.pellegrini@labri.fr>

[NAME](#) | [SYNOPSIS](#) | [DESCRIPTION](#) | [OPTIONS](#) | [TARGET ARCHITECTURES](#) | [MAPPINGS](#) | [EXAMPLES](#) | [SEE ALSO](#) | [AUTHOR](#)

Want to link to this manual page? Use this URL:

<<https://man.freebsd.org/cgi/man.cgi?query=gpart&manpath=FreeBSD+15.1-RELEASE+and+Ports.quarterly>>

[home](#) | [help](#)

[Legal Notices](#) | © 1995-2026 The FreeBSD Project. All rights reserved.

[Contact](#)